



Southeast University

Introduction to Software Engineering

Wang Lulu, Liao Li

lliao@seu.edu.cn

Outline

Part 0. Overview

Part 1. Software Processes

Part 2. Modeling

❖ **Chap 7. Understanding Requirements**

❖ **Chap 8-10. Requirements Modeling**

❖ **Chap 11. Design Concepts**

❖ **Chap 12-14. Design Methods**

Part 3. Quality Assurance

Part 4. Software Project Management

Chap 11. Design Concepts

- ❖ **1. Design**
- ❖ **2. Design and quality**
- ❖ **3. Design Principles**
- ❖ **4. Fundamental Concepts**
 - **Abstraction**
 - **Refinement**
 - **Architecture**
 - **Patterns**
 - **Modularity**
 - **Hiding**
 - **Functional independence**
 - **Aspects**
 - **Refactoring**
 - **OO design concepts**
 - **Design Classes**

Design

- Mitch Kapor, Lotus 1-2-3的创造者, 在《Dr. Dobbs Journal》上提出了一个“软件设计宣言”。他说:

- 好的软件设计应展示:

❖ **Mitch Kapor, the creator of Lotus 1-2-3, presented a “software design manifesto” in *Dr. Dobbs Journal*. He said:**

- **Good software design should exhibit:**
- ***Firmness*: A program should not have any bugs that inhibit its function.**
- ***Commodity*: A program should be suitable for the purposes for which it was intended.**
- ***Delight*: The experience of using the program should be pleasurable one.**

- **`Firmness` (坚固性) : 程序不应有妨碍其功能的错误。**

- **`Commodity` (适用性) : 程序应适合其预定用途。**

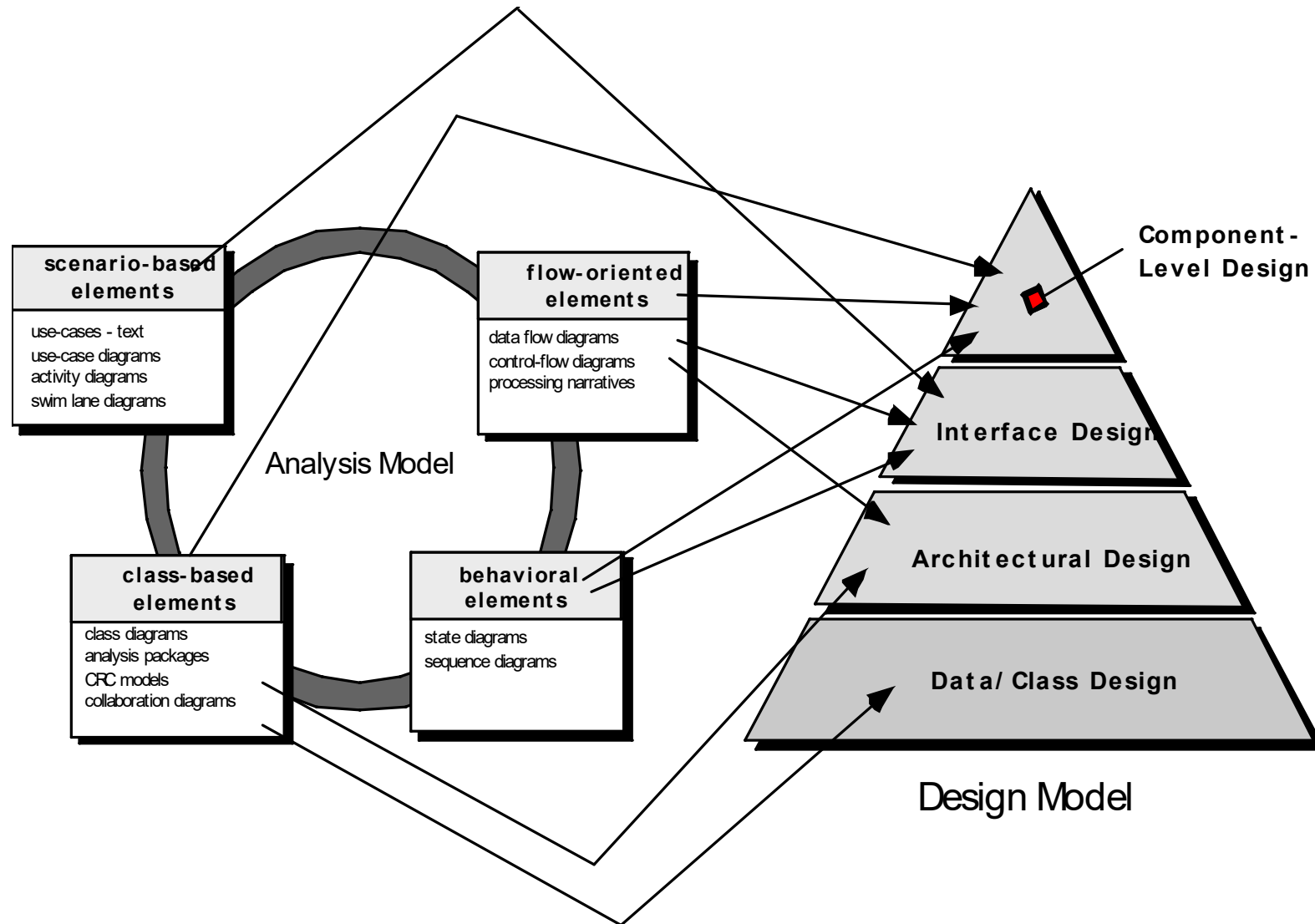
- **`Delight` (愉悦性) : 使用程序的体验应是愉悦的。**

Software Design

- ❖ **Design Engineering creates a representation or model of the software.**
- ❖ **Analysis models focus on describing required data, function and behavior.**
- ❖ **Design models provide detail about software data structure, architecture, interfaces and components.**

- 设计工程创建软件的表示或模型。
- 分析模型侧重于描述所需的数据、功能和行为。
- 设计模型提供关于软件数据结构、架构、接口和组件的详细信息。——

Analysis Model -> Design Model



Software Engineering Design

- ❖ **Data/Class design** – transforms analysis classes into implementation classes and data structures
- ❖ **Architectural design** – defines relationships among the major software structural elements
- ❖ **Interface design** – defines how software elements, hardware elements, and end-users communicate
- ❖ **Component-level design** – transforms structural elements into procedural descriptions of software components

2. 软件工程设计

- Data/Class design——将分析类转换为实现类和数据结构
- Architectural design——定义主要软件结构元素之间的关系
- Interface design——定义软件元素、硬件元素和最终用户如何通信
- Component-level design——转换结构元素到软件组件的过程描述中

设计和质量

Design and Quality

- ❖ **the design must implement all of the explicit requirements contained in the analysis model, and it must accommodate all of the implicit requirements desired by the customer.**
- ❖ **the design must be a readable, understandable guide for those who generate code and for those who test and subsequently support the software.**
- ❖ **the design should provide a complete picture of the software, addressing the data, functional, and behavioral domains from an implementation perspective.**

- 良好设计演化的特征

1. 设计必须实现分析模型中包含的所有显式需求，并且必须满足客户所需的所有隐式需求
2. 对于生成代码的人和测试并随后支持软件的人来说，设计必须是可读的、可理解的指南
3. 设计应提供软件的完整图景，从实现的角度解决数据、功能和行为领域

质量指导原则

Quality Guidelines

- ❖ **A design should exhibit an architecture that**
 - (1) has been created using recognizable architectural styles or patterns,
 - (2) is composed of components that exhibit good design characteristics and
 - (3) can be implemented in an evolutionary fashion
- ❖ **A design should be modular; that is, the software should be logically partitioned into elements or subsystems**
- ❖ **A design should contain distinct representations of data, architecture, interfaces, and components.**

1. 架构

- 是使用可识别的架构风格或模式创建的
 - 由表现出良好设计特征的组件组成
 - 可以以进化的方式实现
- 2. 模块化——软件应在逻辑上划分为元素或子系统
 - 3. 不同表示——含数据、架构、接口和组件

Quality Guidelines

- ❖ **A design should lead to data structures that are appropriate for the classes to be implemented and are drawn from recognizable data patterns.**
- ❖ **A design should lead to components that exhibit independent functional characteristics.**
- ❖ **A design should lead to interfaces that reduce the complexity of connections between components and with the external environment.**
- ❖ **A design should be derived using a repeatable method that is driven by information obtained during software requirements analysis.**
- ❖ **A design should be represented using a notation that effectively communicates its meaning.**

4. 导出数据结构——适用于要实现的类的数据结构，并从可识别的数据模式中提取

5. 导出显示独立功能特征的构件

6. 导出接口——降低组件之间以及与外部环境的连接复杂性

7. 可重复——使用由软件需求分析期间获得的信息驱动的可重复方法推导出设计

8. 可读性——使用有效传达其含义的符号来表示设计

Design Principles

- ❖ The design process should not suffer from ‘tunnel vision.’
- ❖ The design should be traceable to the analysis model.
- ❖ The design should not reinvent the wheel.
- ❖ The design should “minimize the intellectual distance” [DAV95] between the software and the problem as it exists in the real world.
- ❖ The design should exhibit uniformity (一致性) and integration.
- ❖ The design should be structured to accommodate change.
- ❖ The design should be structured to degrade gently, even when aberrant data, events, or operating conditions are encountered.
- ❖ Design is not coding, coding is not design.
- ❖ The design should be assessed for quality as it is being created, not after the fact.
- ❖ The design should be reviewed to minimize conceptual (semantic) errors.

- 设计过程不应受到“隧道视野”的影响。

- 设计应可追溯到分析模型。

- 设计不应重新发明轮子。

- 设计应“最小化软件与现实世界中存在的问题之间的智力距离”[DAV95]。

- 设计应表现出一致性（一致性）和集成性。

- 设计应结构化以适应变化。

- 设计应结构化以温和降级，即使遇到异常数据、事件或操作条件。

- 设计不是编码，编码也不是设计。

- 设计应在创建过程中进行质量评估，而不是事后。

- 设计应经过审查以尽量减少概念（语义）错误。

Fundamental Concepts

- ❖ **Abstraction**—data, procedure
- ❖ **Refinement**—elaboration of detail for all abstractions
- ❖ **Architecture**—the overall structure of the software
- ❖ **Patterns**—“conveys the essence” of a proven design solution
- ❖ **Separation of concerns**—any complex problem can be more easily handled if it is subdivided into pieces
- ❖ **Modularity**—compartmentalization of data and function
- ❖ **Hiding**—controlled interfaces
- ❖ **Functional independence**—single-minded function and low coupling
- ❖ **Aspects**—a mechanism for understanding how global requirements affect design
- ❖ **Refactoring**—a reorganization technique that simplifies the design
- ❖ **OO design concepts**—Appendix II
- ❖ **Design Classes**—provide design detail that will enable analysis classes to be implemented

Fundamental Concepts

- **抽象**——数据，过程
- **细化**——对所有抽象的细节进行详细说明
- **架构**——软件的整体结构
- **模式**——“传达已验证设计解决方案的本质”
- **关注点分离**——任何复杂问题都可以通过分成多个部分更容易处理
- **模块化**——数据和功能的分隔
- **隐藏**——控制的接口
- **功能独立性**——单一功能和低耦合
- **方面**——一种理解全局需求如何影响设计的机制
- **重构**——简化设计的重组技术
- **面向对象设计概念**——附录II
- **设计类**——提供使分析类能够实现的设计细节

Abstraction



中国软件大会(CCF ChinaSoft)

第20届全国软件与应用学术会议

The 20th National Software and Application Conference

第6届全国形式化方法与应用会议

The 6th National Conference on Formal Method and Application

抽象对计算机系统设计至关重要

Hints and Principles for Computer System Design

Butler Lampson

May 14, 2021

2.1 Abstraction—Have a spec

The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise. —Edsger Dijkstra^[2]

Without a specification, a system cannot be wrong, it can only be surprising. —Gary McGraw^[2]

If you're not writing a program, don't use a programming language. —Leslie Lamport^[2]

The hardest part ... is arriving at a complete and consistent specification, and much of the [work is] the debugging of the specification. —Fred Brooks^[2]

Architecture is the things about a computer that a machine language programmer must understand to write a correct (timing independent) program for the machine. specification —Gene Amdahl at AT^[2]

Spec == code

{Keep secrets. Free the implementer.
Good fences make good neighbors.
Embrace nondeterminism. Abstractions leak.

Abstraction is the most important idea in computing. It's the way to make things simple enough that your limited brain can get the machine to do what you want, even though the details of what it does are too complicated for you to track: many, many steps and many, many bits of data. The idea is to have a specification for the computer system that tells its clients

- what, everything they need to know to use the system,
- but not how: nothing about how it works inside—the code ("implementation" is too long).



1992年图灵奖得主Butler Lampson

抽象是计算系统设计中最重要思想。

抽象可以使系统得到简化。

尽管充满繁复的计算过程和大量的数据，
但通过抽象，计算系统可以被简化到按
操作者大脑中的想法，被自由地控制运行。

第20届全国软件与应用学术会议

The 20th National Software and Application Conference

第6届全国形式化方法与应用会议

The 6th National Conference on Formal Method and Application

2021年5月26日

孙凝晖

中国工程院院士



Abstraction

❖ There are many levels of abstraction

- The highest level of abstraction: a solution in broad terms
- Lower levels of abstraction: a more detailed description of the solution.

- 抽象有许多层次：
 - 最高层次的抽象：以宽泛的术语提供解决方案
 - 较低层次的抽象：更详细地描述解决方案

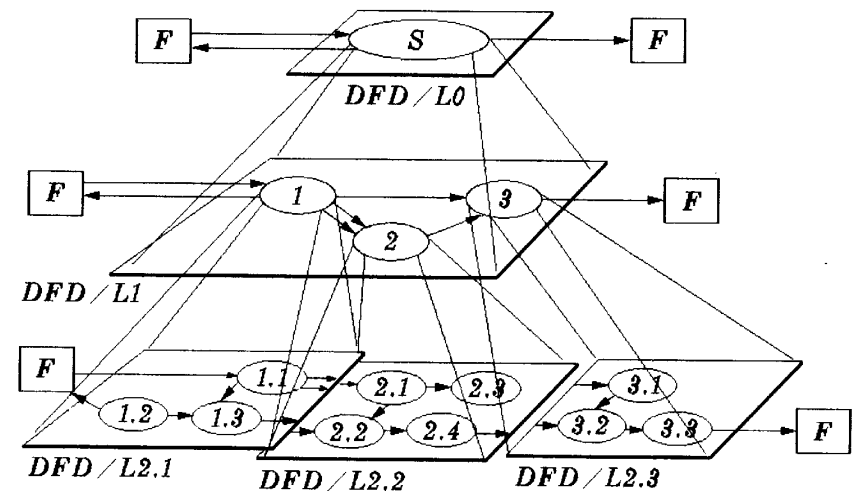
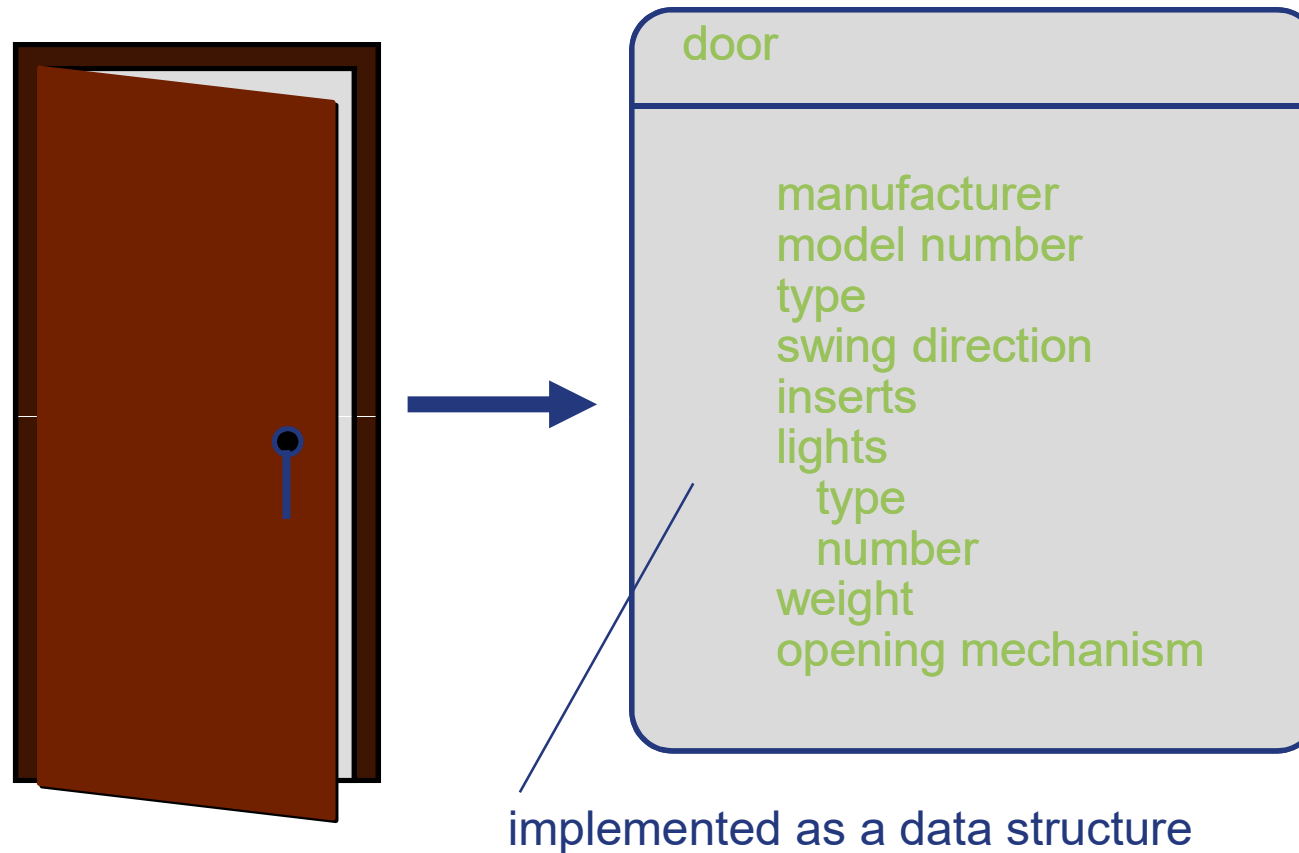
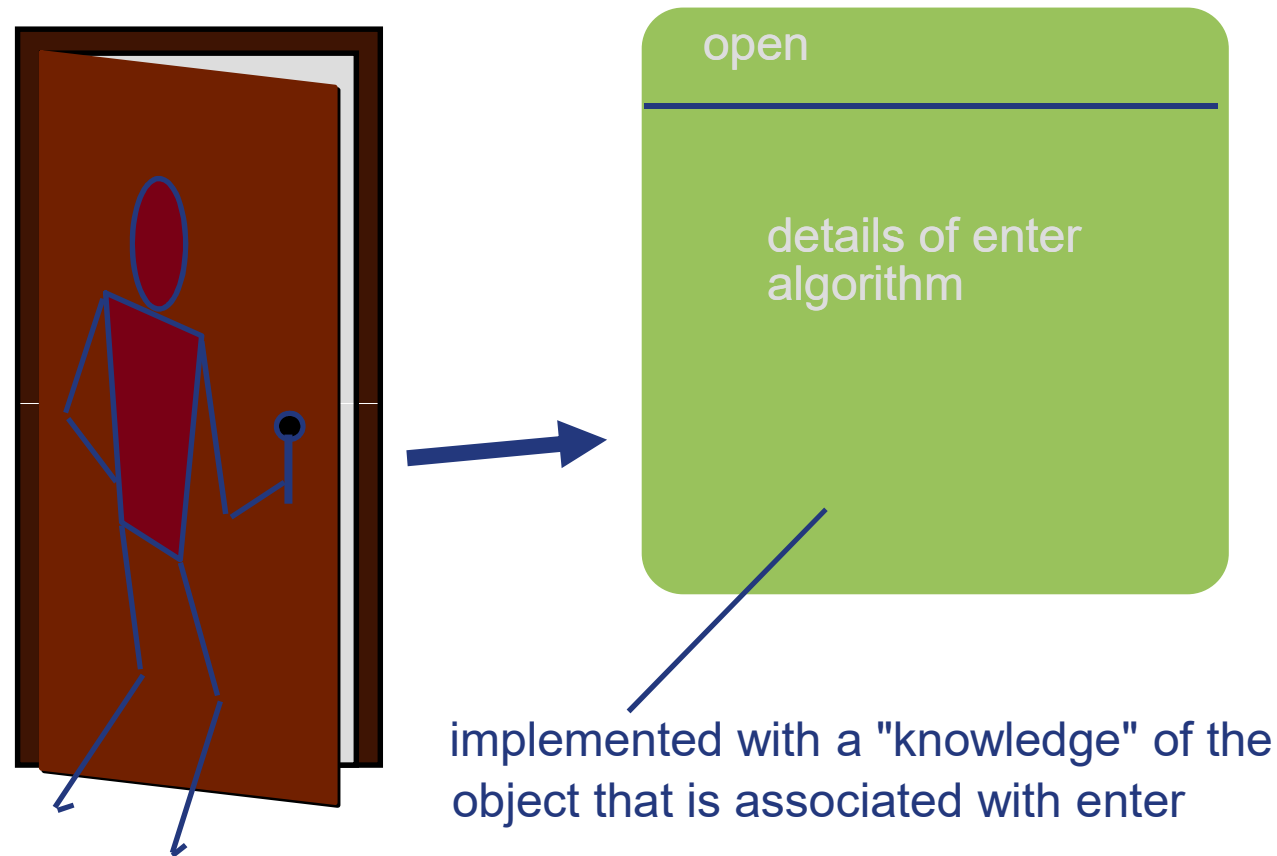


图 2.9 分层数据流图

Data Abstraction



Procedural Abstraction

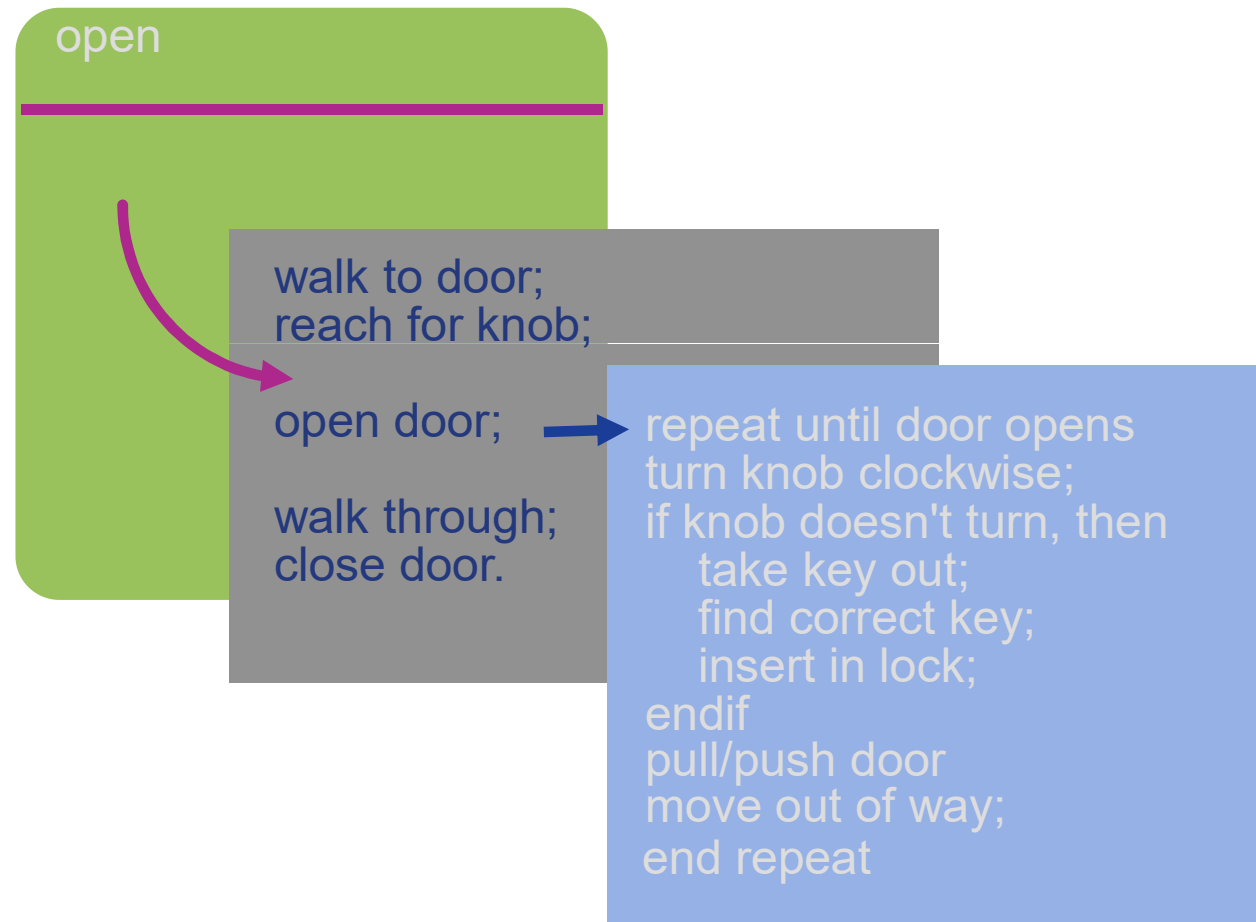


逐步细化

Stepwise Refinement

- ❖ **Stepwise refinement is a top-down design strategy.**
- ❖ **Refinement is actually a process of elaboration.**
- ❖ **Abstraction and refinement are complementary concepts.**
- 逐步细化是一种自顶向下的设计策略。
- 细化实际上是一个详细说明的过程。
- 抽象和细化是互补的概念。

Stepwise Refinement



Architecture

❖ “The overall structure of the software and the ways in which that structure provides conceptual integrity for a system.” [SHA95a]

- **Structural properties:** defines the components of a system (e.g., modules, objects, filters) and the manner in which those components are packaged and interact with one another.

- **Extra-functional properties.** address how the design architecture achieves requirements for performance, capacity, reliability, security, adaptability, and other system characteristics.

- **Families of related systems.** repeatable patterns that are commonly encountered in the design of families of similar systems.

架构

- 软件的整体结构以及该结构如何为系统提供概念上的完整性。
 - 结构属性：定义系统的组件（例如，模块，对象，过滤器）以及这些组件的包装和相互作用方式。
 - 额外功能属性：解决设计架构如何满足性能、容量、可靠性、安全性、适应性等要求。
 - 相关系统的家族：设计类似系统家族时常遇到的可重复模式。

■ 架构的组成

- 结构特性——定义系统的组件（例如，模块、对象、过滤器）以及这些组件的打包方式和相互交互的方式
- 功能外的属性——说明设计架构如何达到对性能、容量、可靠性、安全性、适应性和其他系统特性的要求
- 相关系统——在类似系统系列的设计中经常遇到的可重复模式

■ 架构的作用

- 促进利益相关者的沟通
- 有利于系统设计的早期决策
- 易于理解的系统级抽象

■ 架构体现了系统的

- 模块化
- 抽象和信息隐藏
- 接口设计

Architecture

❖ The role of Architecture

- Facilitate the communication of stakeholders
- Beneficial to the early decision of system design
- Transitive and easy to understand the system level abstraction

架构的角色

- 促进利益相关者之间的沟通
- 有助于系统设计的早期决策
- 传递并易于理解系统级抽象

Architecture

- ❖ The architecture reflects the high-level scheme of software system
 - Software system is becoming more and more complex, which needs to be divided into several parts (**modularization**)
 - Different groups or developers are responsible for different parts
 - then integrate the parts at the system level
 - Developers in charge of different parts need to know less about other modules (**abstraction and information hiding**)
 - These parts also need to define a clear interface (**interface design**)

Architecture

- 架构反映了软件系统的高级方案
 - 软件系统变得越来越复杂，需要分成多个部分（模块化）
 - 不同的组或开发人员负责不同的部分
 - 然后在系统级别集成这些部分
 - 负责不同部分的开发人员需要了解其他模块的较少信息（抽象和信息隐藏）
 - 这些部分还需要定义一个清晰的接口（接口设计）

Pattern

❖ **Each pattern describes a recurring problem in our environment and describes the core solution to the problem.**

❖ **The goal of a pattern: easy to reuse.**

❖ **Types of patterns**

- **Architecture Pattern**

- **Design Pattern**

- **Idiom**

- 每种模式描述了我们环境中的一个反复出现的问题，并描述了该问题的核心解决方案。
- 模式的目标：易于重用。
- 模式的类型：
 - 架构模式
 - 设计模式
 - 惯用法

Architectural Patterns

- ❖ **The basic structure organization form or structure scheme of software system is expressed.**
- ❖ **The Architectural Pattern consists of a set of predefined subsystems that define their responsibilities, and provides rules and guides for organizing and managing these subsystems.**

- **E.g. B/S, C/S**

- 表达软件系统的基本结构组织形式或结构方案。
- 架构模式由一组预定义的子系统组成，这些子系统定义了它们的职责，并提供了组织和管理这些子系统的规则和指南。

- 例如，B/S，C/S

Design Patterns

- ❖ **It provides a refined solution for the subsystem, the component or the relationship between components of a software system.**
- ❖ **It describes the components used to solve common software design problems in a specific environment and the reproducible structure of these components when they communicate with each other.**

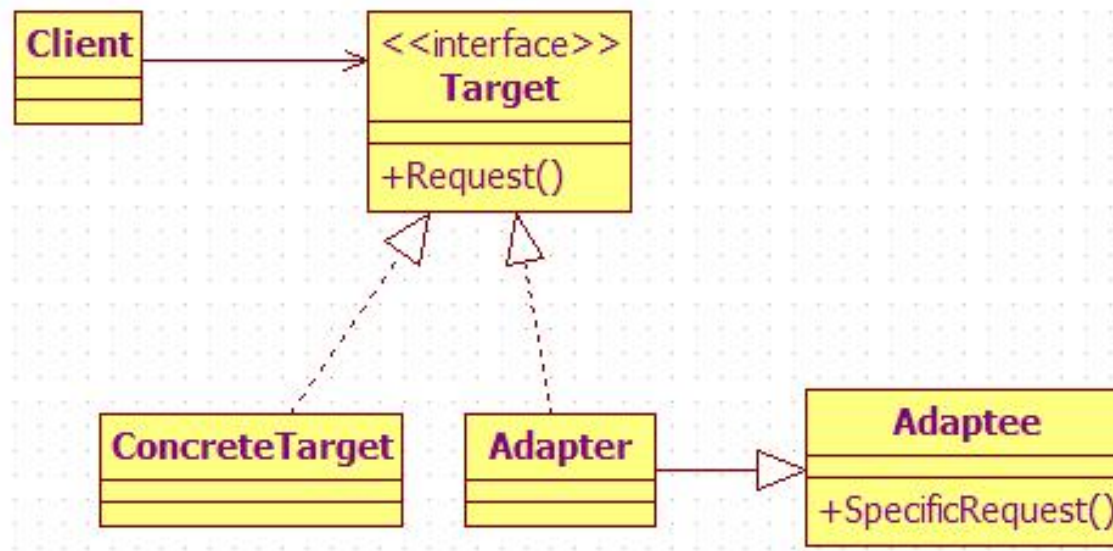
设计模式

- 提供子系统、组件或软件系统组件之间关系的精细解决方案。
- 描述用于解决特定环境中常见软件设计问题的组件，以及这些组件在相互通信时的可复制结构。

Design Patterns

- ❖ The **Adapter** mode allows things that could not work together because of interface incompatibility to work together.

适配器模式允许由于接口不兼容而无法一起工作的事物一起工作。



Design Patterns

Design Pattern Template

Pattern name—describes the essence of the pattern in a short but expressive name

Intent—describes the pattern and what it does

Also-known-as—lists any synonyms for the pattern

Motivation—provides an example of the problem

Applicability—notes specific design situations in which the pattern is applicable

Structure—describes the classes that are required to implement the pattern

Participants—describes the responsibilities of the classes that are required to implement the pattern

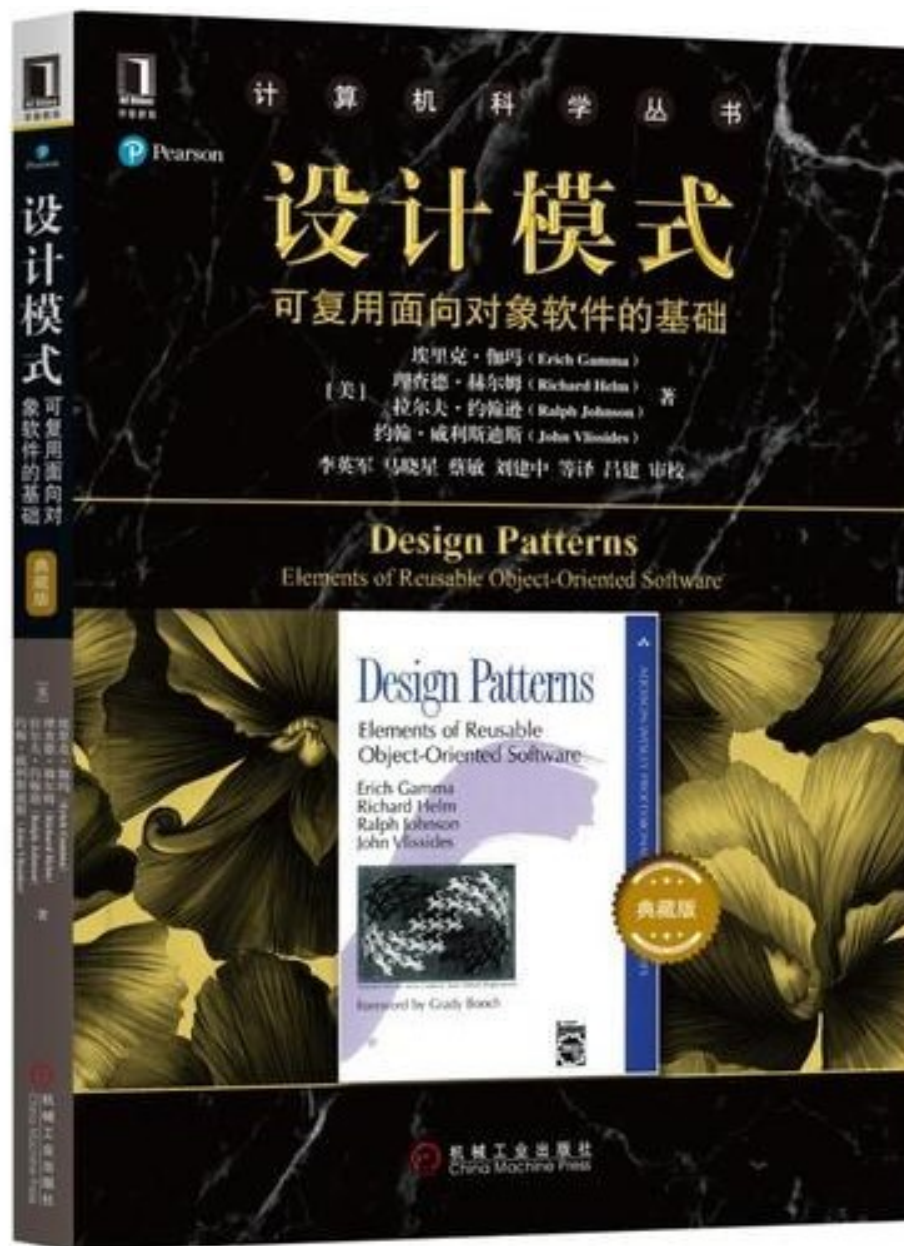
Collaborations—describes how the participants collaborate to carry out their responsibilities

Consequences—describes the “design forces” that affect the pattern and the potential trade-offs that must be considered when the pattern is implemented

Related patterns—cross-references related design patterns

Design Patterns

- 设计模式模板
 - 模式名称——用简短但有表现力的名称描述模式的本质
 - 意图——描述模式及其功能
 - 也称为——列出模式的任何同义词
 - 动机——提供问题的示例
 - 适用性——记录模式适用的特定设计情况
 - 结构——描述实现模式所需的类
 - 参与者——描述实现模式所需类的职责
 - 协作——描述参与者如何合作以履行职责
 - 结果——描述影响模式的“设计力量”以及在实施模式时必须考虑的潜在权衡
 - 相关模式——交叉引用相关设计模式



Idiom

- ❖ **An idiom is a low-level pattern specific to a programming language.**
- ❖ **An idiom describes how to implement particular aspects of components or the relationships between them using the features of the given language.**

- 惯用法是特定于编程语言的低级模式。
- 惯用法描述了如何使用特定语言的特性来实现组件的特定方面或它们之间的关系。

5 变量

15-1: 去掉没必要的公共变量。	是[] 否[] 免 []	
15-2: 仔细定义并明确公共变量的含义、作用、取值范围及公共变量间的关系。		
15-3: 明确公共变量与操作此公共变量的函数或过程的关系，如访问、修改及创建等。		
15-4: 当向公共变量传递数据时，要十分小心，防止赋与不合理的值或越界等现象发生。		
15-5: 防止局部变量与公共变量同名。		
15-6: 严禁使用未经初始化的变量作为右值。		

6 函数、过程

16-1: 对所调用函数的错误返回码要仔细、全面地处理。	是[] 否[] 免 []	
16-2: 明确函数功能，精确（而不是近似）地实现函数设计。	是[] 否[] 免 []	

分离关注点

Separation of Concerns

- ❖ Any complex problem can be more easily handled if it is subdivided into pieces that can each be solved and/or optimized independently.
- ❖ A *concern* is a feature or behavior that is specified as part of the requirements model for the software.
- ❖ By separating concerns into smaller, and therefore more manageable pieces, a problem takes less effort and time to solve.

- 任何复杂问题都可以更容易处理，如果它被分成可以独立解决和/或优化的部分。

- 关注点是作为软件需求模型的一部分而指定的功能或行为。

- 通过将关注点分成更小、更易管理的部分，问题的解决将花费更少的精力和时间。

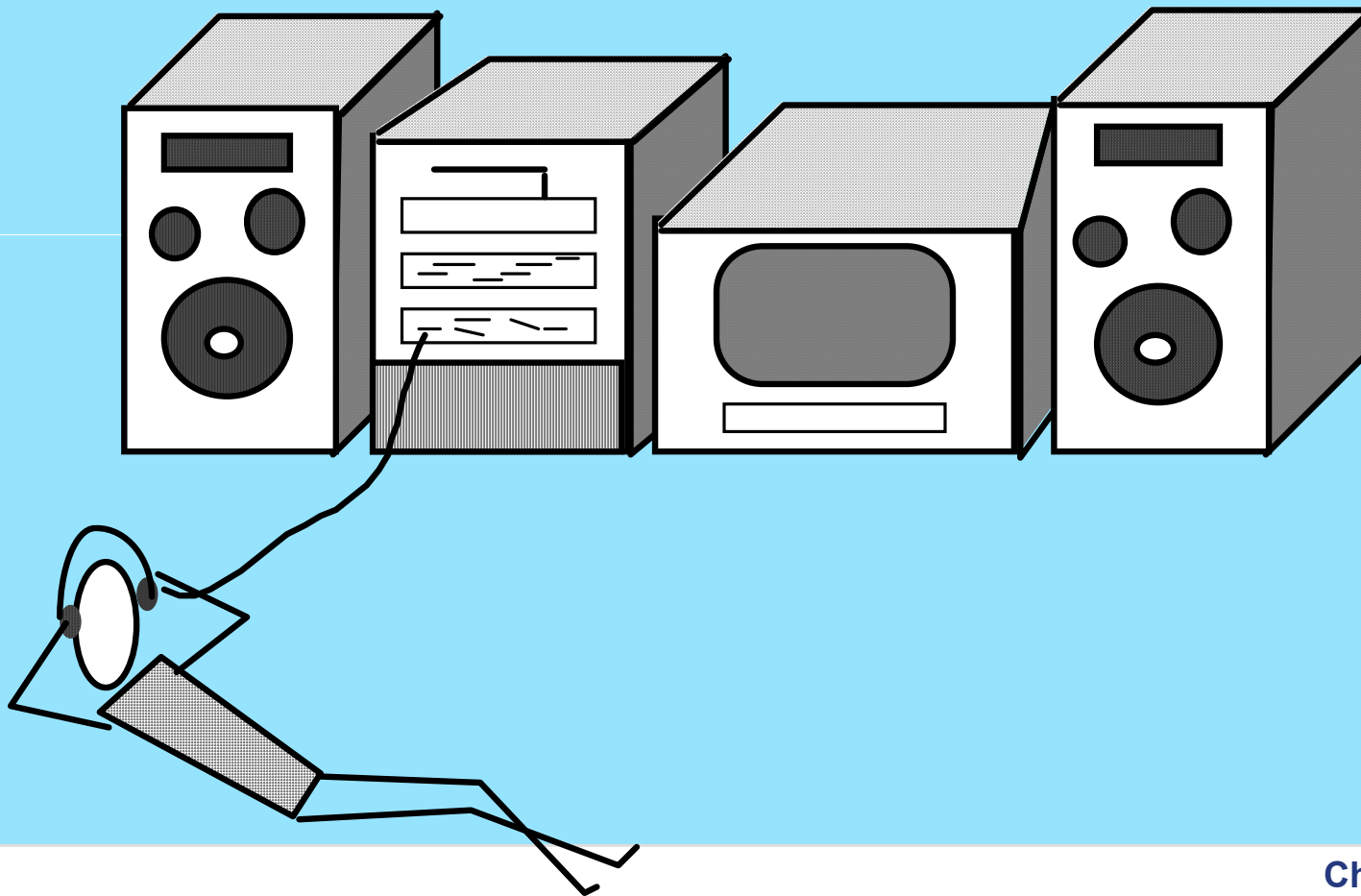
Modularity

- ❖ "modularity is the single attribute of software that allows a program to be intellectually manageable" [Mye78].
- ❖ Monolithic software (i.e., a large program composed of a single module) **cannot** be easily grasped by a software engineer.
 - The number of control paths, span of reference, number of variables, and overall complexity would make understanding close to impossible.
- ❖ In almost all instances, you should break the design into many modules, hoping to make understanding easier and as a consequence, reduce the cost required to build the software.

- 模块化是软件的单一属性，使程序能够在智力上易于管理。
- 单片软件（即，由一个单一模块组成的大程序）不能被软件工程师轻易掌握。
 - 控制路径数量、参考跨度、变量数量和整体复杂性将使理解几乎变得不可能。
- 在几乎所有情况下，你都应该将设计分解成多个模块，以便使理解更容易，从而减少构建软件所需的成本。

Modularity

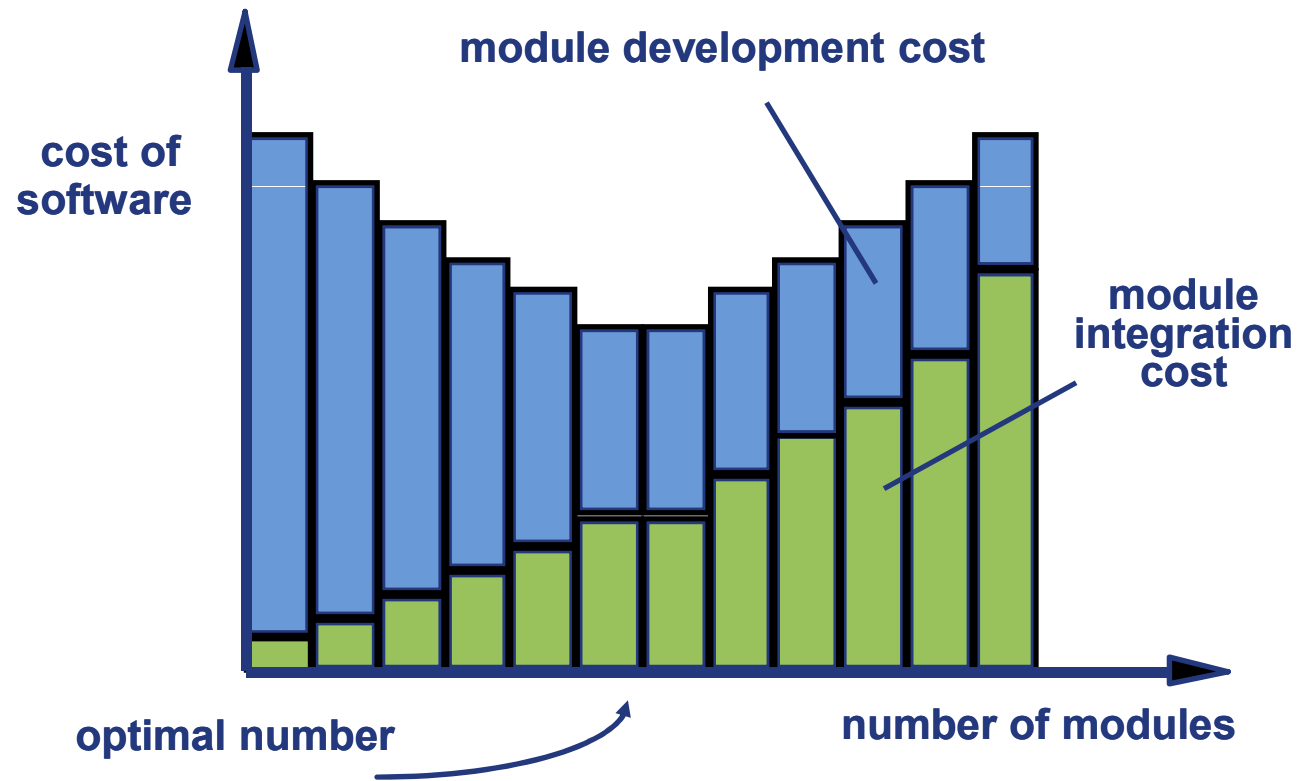
easier to build, easier to change, easier to fix ...



Modularity: Trade-offs

- 模块化——数据和功能的划分
 - 模块集成的成本使得模块并非越小越好

What is the "right" number of modules for a specific software design?





中国软件大会(CCF ChinaSoft)

第20届全国软件与应用学术会议

The 20th National Software and Application Conference

第6届全国形式化方法与应用会议

The 6th National Conference on Formal Method and Application

分解粒度是复杂系统设计的关键抽象

砖石建筑设计



分解粒度：砖头

- 统一基础单元，是一种低层次抽象
- 忽略系统中各部分的结构和功能差异，分解易，组合难
- **组合仍然是复杂优化系统**

预制板建筑设计



分解粒度：预制件

- 不同规格、不同特征的功能模组，是一种高层次抽象
- 根据系统中各部分的结构和功能差异，充分考虑可配置性和可组合性，分解难，组合易
- **预制件功能模组是结构化简单系统**

8

第20届全国软件与应用学术会议

The 20th National Software and Application Conference

第6届全国形式化方法与应用会议

The 6th National Conference on Formal Method and Application

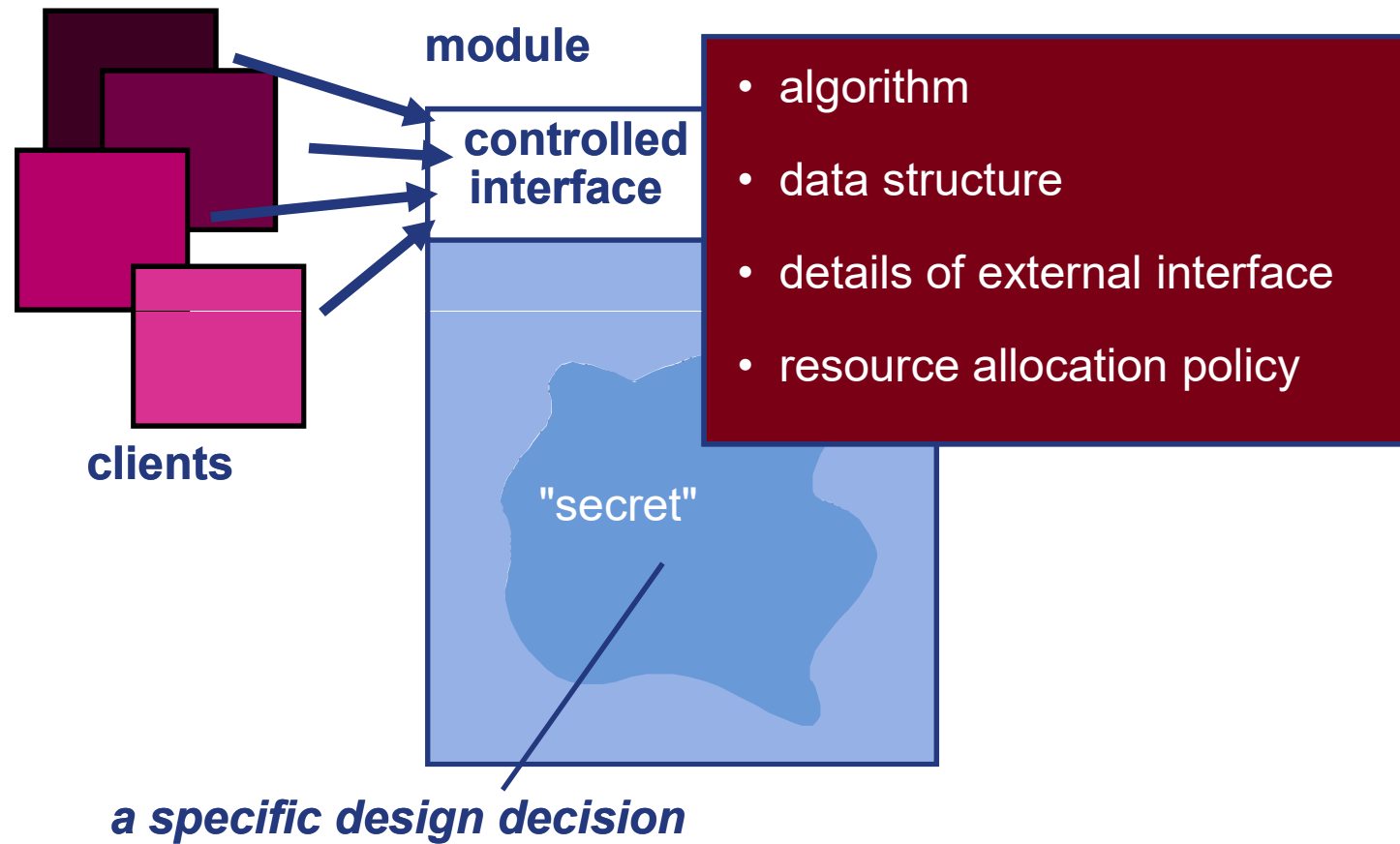
2021年12月15-16日

孙凝晖

中国工程院院士



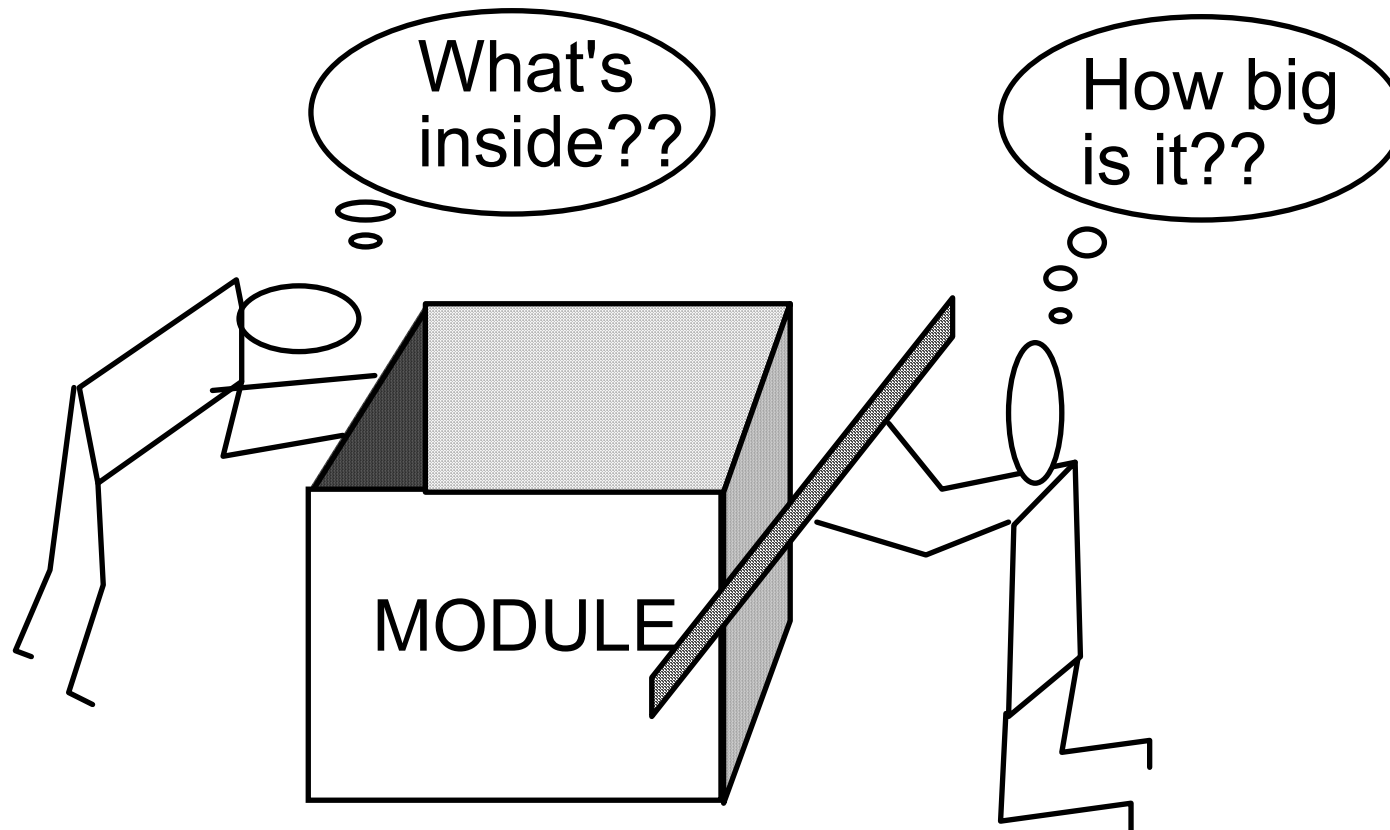
Information Hiding



Why Information Hiding?

- ❖ **reduces the likelihood of “side effects”**
- ❖ **limits the global impact of local design decisions**
 - 减少“副作用”的可能性
 - 限制局部设计决策的全球影响
- ❖ **emphasizes communication through controlled interfaces**
 - 强调通过受控接口进行通信
 - 不鼓励使用全局数据
- ❖ **discourages the use of global data**
 - 导致封装——高质量设计的一个属性
 - 产生更高质量的软件
- ❖ **leads to encapsulation—an attribute of high quality design**
- ❖ **results in higher quality software**

Functional Independence



Sizing Modules: Two Views

Functional Independence

- 通过开发具有“单一功能”和对过多与其他模块的交互“厌恶”的模块来实现功能独立性。
 - ❖ **Functional independence is achieved by developing modules with “single-minded” function and an “aversion (厌恶)” to excessive interaction with other modules.**
 - ❖ **Cohesion** is an indication of the relative functional strength of a module.
 - A cohesive module performs a single task, requiring little interaction with other components in other parts of a program. Stated simply, a cohesive module should (ideally) do just one thing.
 - ❖ **Coupling** is an indication of the relative interdependence among modules.
 - Coupling depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface.
- 内聚是模块相对功能强度的一个指标。
 - 一个内聚模块执行单一任务，需要与程序中其他部分的其他组件进行较少的交互。简单来说，一个内聚模块应该（理想情况下）只做一件事。
 - 耦合是模块之间相对相互依赖性的一个指标。
 - 耦合取决于模块之间接口的复杂性、对模块进行条目或引用的点以及跨接口传递的数据。

Functional Independence

❖ 1) **Function independence (module independence) is the fundamental requirement of modularization**

- **Modules perform independent functions: clear and identifiable**
- **High cohesion**: tight internal structure
- **Low coupling**: the degree of correlation and dependence between modules is as small as possible, and the interface with other modules is simple
- **It accords with the principle of information concealment (信息隐蔽) and information localization (信息局部化)**

1) 功能独立性（模块独立性）是模块化的基本要求

- 模块执行独立功能：清晰且可识别
- 高内聚：内部结构紧密
- 低耦合：模块之间的关联和依赖程度尽可能小，模块与其他模块的接口简单
- 符合信息隐藏（信息隐藏）和信息局部化（信息局部化）的原则

功能独立性的的重要性

Functional Independence

❖ 2) The importance of Functional Independence

- Module developers focus on a relatively independent part
- They need not to worry too much about other modules
- **The scope of the impact of modifications and bugs is localized**
- Single module is **easier to reuse**
- Independent modules are **easier to maintain and test**

- 模块开发人员专注于相对独立的部分
- 他们不需要太担心其他模块
- 修改和错误的影响范围是局部化的
- 单一模块更容易重用
- 独立模块更容易维护和测试

Functional Independence

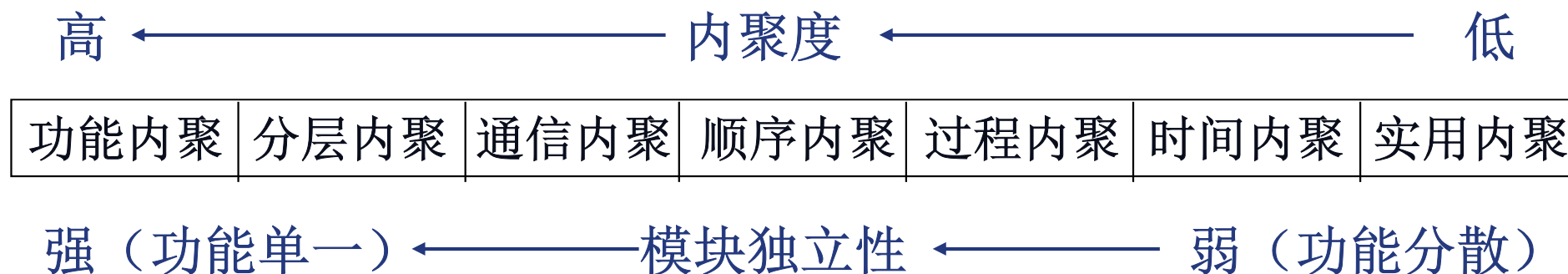
❖ 3) 模块独立性的指标

- 内聚度与耦合度
- 内聚(**cohesion**): 一个模块内部各个元素彼此结合的紧密程度
— 尽量高
- 耦合(**coupling**): 模块之间相互关联的程度
— 尽量低

Functional Independence

❖ 3) 模块独立性的指标

- 内聚度（由高到低排列）



Functional Independence

❖ 3) 模块独立性的指标:内聚度

■ 功能内聚 (Functional cohesion)

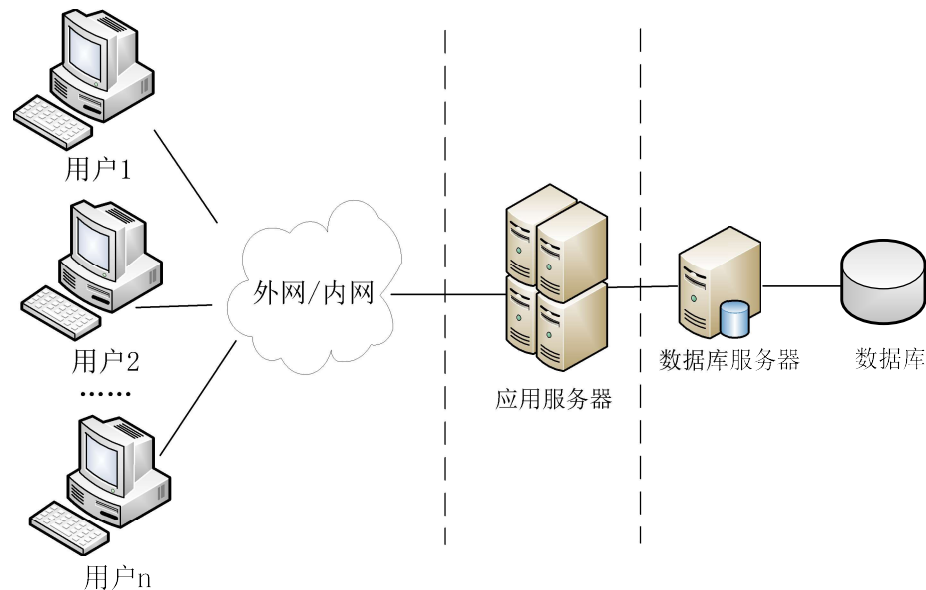
- 指一个模块中各个部分都是为完成一项具体功能而协同工作，紧密联系，不可分割的(单个功能)
- 判断一个模块是不是功能内聚：
 - 若需要完成的功能中含“和”，“或”等词，它一定不够功能内聚。

Functional Independence

❖ 3) 模块独立性的指标:内聚度

■ 分层内聚 (Layer cohesion)

- 层是一种弱耦合结构，层与层之间的依赖是向下的，底层对于上层而言是“无知”的，改变上层的设计对于其调用的底层而言没有任何影响。



Functional Independence

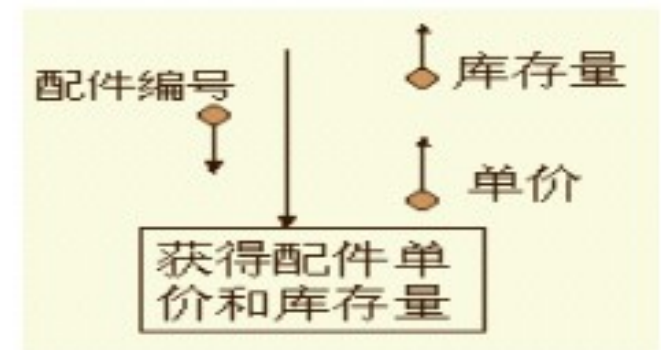
❖ 3) 模块独立性的指标:内聚度

■ 顺序内聚 (Sequential cohesion)

- 一个模块内部各个组成部分必须顺序执行，前一个动作的输出是后一个动作的输入

■ 通信内聚 (communicational cohesion)

- 模块内各组成部分的处理动作都使用相同的输入数据或者具有相同的输出数据

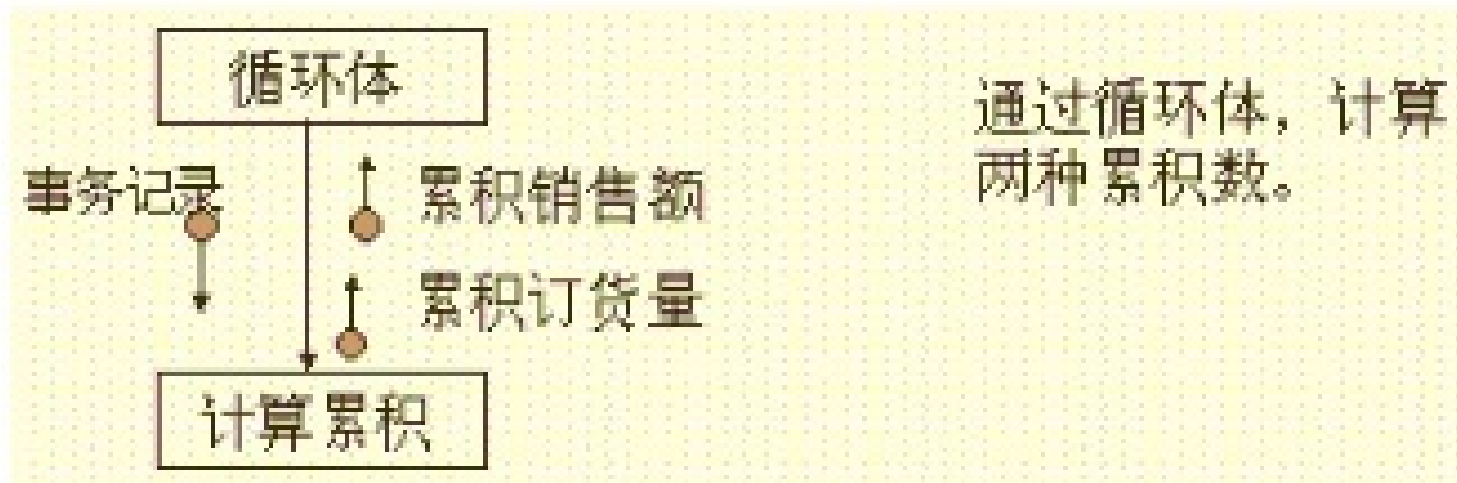


Functional Independence

❖ 3) 模块独立性的指标:内聚度

■ 过程内聚 (Procedural cohesion)

- 模块内的多个任务，即使动作各不相同，也没有相互联系，但它们都受一个控制流支配，必须按指定的过程执行



Functional Independence

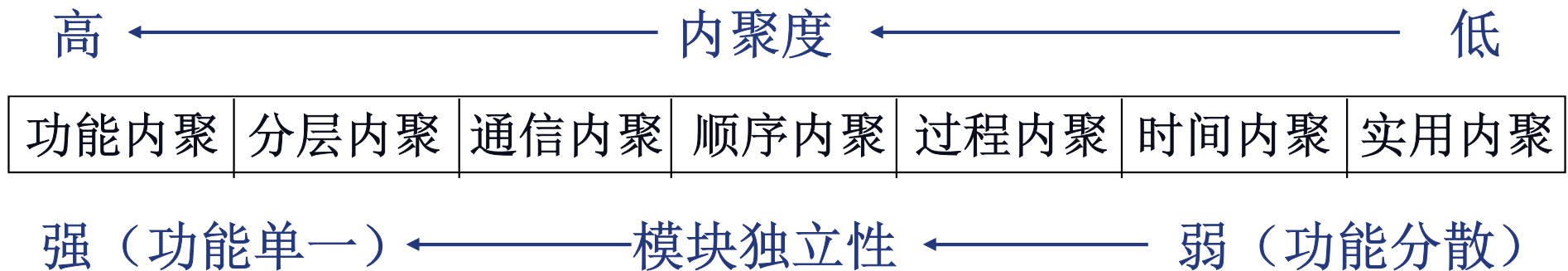
❖ 3) 模块独立性的指标:内聚度

- 暂时内聚 (**Temporal cohesion**, 时间内聚)
 - 模块内各部分的处理与时间相关
 - 时间内聚模块中的各项处理必须在一定时间内完成
 - 如: 启动时要进行的全部操作
- 实用内聚 (**Utility cohesion**)
 - 在一个模块中, 但是其他方面都不相关的构件、类或操作被分为一组。
 - 如: **Statistics**类中包含计算6个简单统计所需的所有属性和操作

Functional Independence

❖ 3) 模块独立性的指标

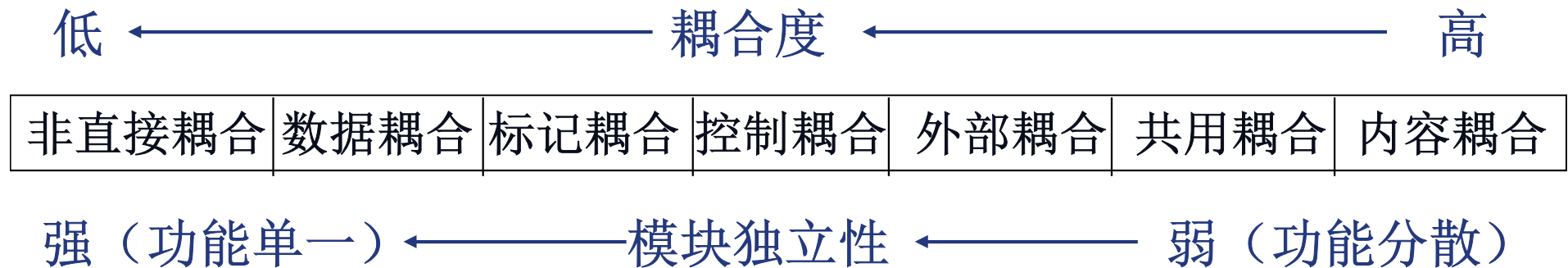
- 内聚度（由高到低排列）



Functional Independence

❖ 3) 模块独立性的指标

- 耦合度（由高到低排列）



Functional Independence

❖ 3) 模块独立性的指标:耦合度

■ 内容耦合 (**Content coupling**)

- 一个模块可以直接访问另一个模块的内部数据或内部功能
- 如:

模块A:

...
L1: ...
...

模块B:

...
GOTO L1
...

Functional Independence

❖3) 模块独立性的指标:耦合度

■ 共用耦合 (Common coupling, 公共耦合)

- 多个模块共同访问某些公共数据环境
- 好处: 便利性 (如可设公共缺省值)
- 缺点: 当这种耦合需要变更时, 无法判定有多少模块属于共用耦合, 从而造成错误蔓延

■ 外部耦合 (External coupling)

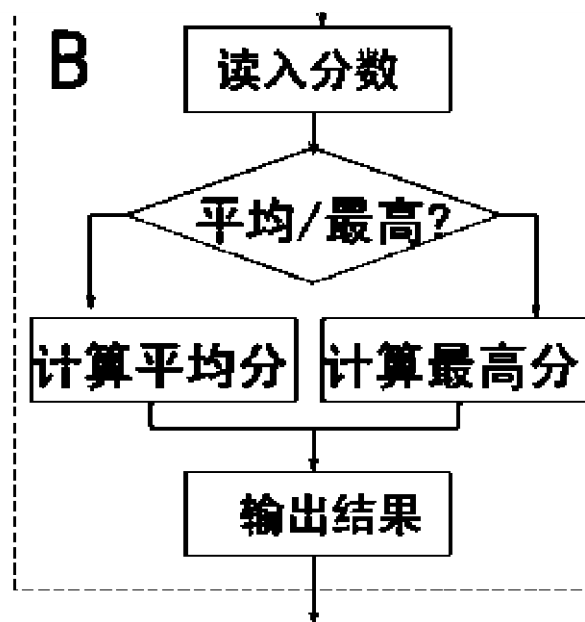
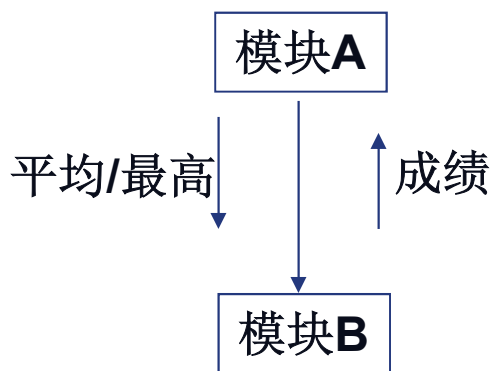
- 一组模块都访问同一全局简单变量而不是同一全局数据结构, 而且不是通过参数表传递该全局变量的信息

Functional Independence

❖ 3) 模块独立性的指标:耦合度

■ 控制耦合 (Control coupling)

- 模块间的交互参数包含控制信息，可影响另一个模块的执行逻辑
- 控制信息不同于数据信息



调用逻辑性模块B时，须先传递控制信号(平均分/最高分)，以选择所需的操作。控制模块必须知道被控模块的内部逻辑，增强了相互依赖。

Functional Independence

❖ 3) 模块独立性的指标:耦合度

■ 标记耦合 (stamp coupling)

- 模块间传递特定的数据结构
- 如高级语言的数组名,记录名,文件名等这些名字 (标记),其实传递的是该数据结构的地址
- 缺点: 各模块都必须清楚该记录的结构, 并按结构要求对该记录进行操作。

■ 数据耦合 (data coupling)

- 模块间仅通过参数表传递简单数据
- 但是要注意若参数列表过长则会带来测试和维护困难。

Functional Independence

❖ 3) 模块独立性的指标:耦合度

- 非直接耦合：两个模块可以相对独立工作
 - 例程调用耦合 (**Routine call coupling**)
 - 一个操作调用另外一个操作
 - 通过减少例程总量来减少此种耦合
 - 类型使用耦合 (**Type use coupling**)
 - 构件A使用了在构件B中定义的一个数据类型
 - 如：一个类将其某变量声明为另一个类的类型
 - 包含或导入耦合 (**Inclusion or import**)
 - 构件A导入或者包含构件B的包或者内容

Functional Independence

- ❖ 具有模块独立性的模块：高内聚低耦合的模块
 - 一个模块内部各个元素之间的联系越紧密，则它的内聚性就越高，模块独立性也越强
 - 模块之间的连接越紧密，联系越多，耦合性就越高，而其模块独立性就越弱

Functional Independence

❖ Decouple（解耦）

- 降低模块间的耦合度，尽可能使得模块之间的耦合是松散耦合。
- 解耦能保持组件之间的自主和独立性，降低维护成本，提高可读性。
- 从宏观角度可以进行架构解耦；
- 从微观角度，对面向对象设计来说，就是尽量减少类之间的连接，给类尽量分配内聚的职责。

Aspects

❖ Consider two requirements, A and B .

Requirement A *crosscuts* requirement B “if a software decomposition [refinement] has been chosen in which B cannot be satisfied without taking A into account. [Ros04]

❖ An *aspect* is a representation of **a cross-cutting concern**.

- 考虑两个需求，A和B。需求A横切需求B，“如果选择了一种软件分解（细化），其中在不考虑A的情况下无法满足B。”[Ros04]
- 方面是交叉关注点的表示。

Refactoring

- Fowler [FOW99]以以下方式定义重构：

- “重构是改变软件系统的过程，使其不改变代码（设计）的外部行为，却改进其内部结构。”

❖ **Fowler [FOW99] defines refactoring in the following manner:**

- **"Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code [design] yet improves its internal structure."**

❖ **When software is refactored, the existing design is examined for**

- **redundancy**
- **unused design elements**
- **inefficient or unnecessary algorithms**
- **poorly constructed or inappropriate data structures**
- **or any other design failure that can be corrected to yield a better design.**

- 当软件被重构时，现有设计会被检查以：
 - 低效或不必要的算法
 - 冗余
 - 构造不良或不适当的数据结构
 - 未使用的设计元素
 - 或任何其他可以纠正以产生更好设计的设计失败



Southeast University



Thank You !

lliao@seu.edu.cn